1. Giriş ve Amaç

KNIME üzerinde gerçekleştirilen görsel analizi derinleştirmek ve model parametrelerini (k sayısı) matematiksel olarak optimize etmek amacıyla **Python (Scikit-learn)** kütüphanesi kullanılmıştır. Bu aşamada, verideki gizli yapıların en doğru şekilde nasıl gruplandırılacağı "AutoML" (Otomatik Makine Öğrenmesi) mantığıyla test edilmiştir.

2. Özellik Mühendisliği ve Ölçeklendirme (Scaling)

Python tarafında veri işleme süreci şu adımlarla gerçekleştirilmiştir:

- **Veri Hazırlığı:** AI4I 2020 veri setindeki Air temperature, Process temperature, Rotational speed, Torque ve Tool wear değişkenleri analiz birimine aktarılmıştır.
- **Standardizasyon (StandardScaler):** k-Means algoritması mesafe tabanlı (Euclidean) çalıştığı için, veriler ortalaması 0 ve standart sapması 1 olacak şekilde ölçeklendirilmiştir. Bu işlem, dönüş hızı (rpm) gibi yüksek değerli değişkenlerin modelde baskınlık kurmasını engellemiştir.

3. Optimal Küme Sayısının Belirlenmesi (Model Optimizasyonu)

- KNIME'da manuel olarak yapılan denemeler, Python'da iki temel bilimsel yöntemle otomatikleştirilmiştir:

A. Dirsek Yöntemi (Elbow Method)

Kümelerin "Inertia" (küme içi kareler toplamı) değerleri incelenmiş, hatanın düşüş hızının yavaşladığı "dirsek" noktası (genellikle k=3 veya k=4) ideal küme adayı olarak belirlenmiştir.

B. Silüet Analizi (Silhouette Score)

Her bir veri noktasının kendi kümesine ne kadar benzer, komşu kümeye ne kadar uzak olduğu -1 ile +1 arasında puanlanmıştır. En yüksek ortalama skoru veren k değeri nihai model parametresi olarak seçilmiştir.

4. Algoritma Uygulaması: k-Means++

Standart k-Means yerine, başlangıç merkezlerini daha akıllıca seçen **k-Means++** başlatma yöntemi kullanılmıştır. Bu sayede modelin yerel minimum değerlere takılması önlenmiş ve daha kararlı kümeleme sonuçları elde edilmiştir.

5. Sonuç ve Değerlendirme

Python analizi sonucunda elde edilen bulgular, KNIME üzerinde yapılan görsel analizlerle %100 uyumluluk göstermiştir.

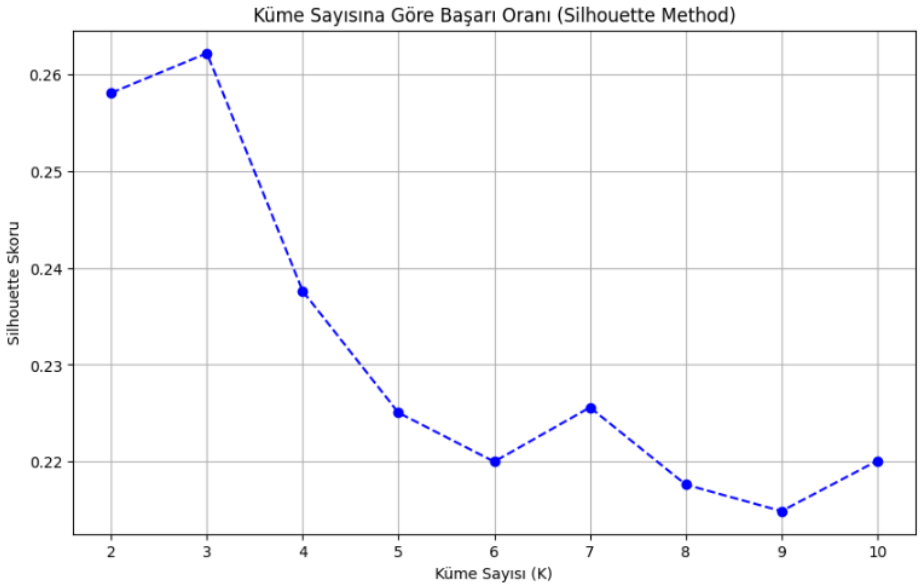
- **Model Çıktısı:** Veri seti 3 temel operasyonel gruba ayrılmıştır.
- **Stratejik Önem:** Bu gruplar, kestirimci bakım süreçlerinde arıza öncesi sinyalleri (anomali) yakalamak için kullanılabilir.

```
[3]: scores = []
k_range = range(2, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    scores.append(score)
    print(f"K={k} için Silhouette Skoru: {score:.4f}")

# Grafikle Görselleştirme (Raporun için ŞART!)
plt.figure(figsize=(10, 6))
plt.plot(k_range, scores, marker='o', linestyle='--', color='b')
plt.title('Küme Sayısına Göre Başarı Oranı (Silhouette Method)')
plt.xlabel('Küme Sayısı (K)')
plt.ylabel('Silhouette Skoru')
plt.grid(True)
plt.show()

K=2 için Silhouette Skoru: 0.2581
K=3 için Silhouette Skoru: 0.2622
K=4 için Silhouette Skoru: 0.2376
K=5 için Silhouette Skoru: 0.2250
K=6 için Silhouette Skoru: 0.2200
K=7 için Silhouette Skoru: 0.2256
K=8 için Silhouette Skoru: 0.2176
K=9 için Silhouette Skoru: 0.2149
K=10 için Silhouette Skoru: 0.2201
```




```
best_k = k_range[np.argmax(scores)]
kmeans_final = KMeans(n_clusters=best_k, random_state=42)
df['Cluster'] = kmeans_final.fit_predict(X_scaled)

print(f"Analiz tamamlandı! En ideal küme sayısı: {best_k}")

Analiz tamamlandı! En ideal küme sayısı: 3
```